

Document Number P1844R0
Date 2019-08-04
Audience LEWGI, SG16
Reply-To Nozomu Katō <noz.ka@akenotsuki.com>
Supersedes [P0169](#)

Enhancement of regex

Table of Contents

- I. [Introduction and Motivation](#)
- II. [Why new syntax option? Why not simply enhance the existing ECMAScript mode?](#)
- III. [Why RegExp of ECMAScript?](#)
- IV. [Scope and Impact on the Standard](#)
- V. [Technical Specifications](#)
- VI. [Relevant Matter](#)
- VII. [References](#)
- VIII. [Appendix](#)

Abstract

- Leave the existing `std::regex` and `std::wregex` as they are. They may have problems, but we should not use our time to make them better and/or convenient.
- Instead, add a new syntax option being compliant to a recent version of ECMAScript keeping the nature of being locale independent, for use with `char8_t`, `char16_t`, or `char32_t` through the template specialization and urge to migrate to them.
- These three specializations are not required necessarily to be implemented separately. It is expected that typical implementations use an internal iterator class template that translates a sequence of UTF-8, UTF-16, or UTF-32 to a sequence of Unicode code points and process that translated sequence. The link to a sample implementation is included.

I. Introduction and Motivation

The default engine of C++'s regex is based on RegExp, regular expression objects of the ECMAScript specification third edition. Although its original RegExp had not been being modified for many years, in these years has been enhanced as follows:

- Unicode flag was introduced in ES6 (ES2015) to handle the surrogate pair correctly,
- Variable length lookbehind assertions since ES2018 (ES9), (not like Perl, but like .Net. No restriction on length of what lookbehind can match),

- Named capture groups since ES2018,
- Dotall flag corresponding to `//s` flag in Perl since ES2018,
- Unicode property escapes since ES2018.

Unfortunately, while C++'s regex supports six regular expression grammars, all are inferior to other languages' regular expression features in richness of available expressions nowadays. In addition, the problem of how to support Unicode is still unsettled. To resolve these problems, or at least to improve the current situation, this paper proposes adding the new syntax option, **ECMAScript2019** to regex.

II. Why new syntax option? Why not simply enhance the existing ECMAScript engine?

Short answer: The ECMAScript engine of C++ has been modified to depend on the locale deeply. The author of this proposal wants to release regex from the locale and to revert it to being locale independent as its original RegExp, then to add new features on it. But doing so involves deprecating some features of the default engine and seems to be difficult.

Long answer: RegExp of ECMAScript is locale independent and treats an input sequence as UTF-16. For example, `/[a-z]/` is always interpreted "any character in the range from U+0061 to U+007A inclusive". This has the benefit of allowing that the set of characters that some character class expression matches can be settled even at compile time. This is true even if the `icase` flag is set, by preparing a reversed case-folding table (while case-folding means converting each of "S", "s", and "Œ" to "s", reversed case-folding here means returning the set of characters that are converted into the same character when case-folding is done, such as converting each of "S", "s", and "Œ" to "SsŒ".)

However, the ECMAScript engine of regex can be modified as being locale sensitive by setting the `collate` flag. If this flag is used with the `icase` flag, the pattern compiler is required to call `std::tolower` and `std::toupper` per character in one character class `[]` for gathering all characters that the character class can match. This clumsiness was filed as [Defect Report 523](#) in 2005 and is still Status Open.

Furthermore, the ECMAScript engine was modified to support the POSIX character class and to change the expressions `\d`, `\D`, `\s`, `\S`, `\w`, and `\W` to be equivalent to some of the POSIX character class. This also made the ECMAScript engine depend on the locale.

Although this paper is not intended to propose making regex **constexpr**, if there is any ECMAScript based engine that inherits the nature of being locale independent from its original RegExp, it might make **constexpr** support easier than now.

The modifications defined in `[re.grammar]` also caused [DR2986](#), [DR2987](#) and they are still unresolved. To fix these issues originating in the modifications, I considered to propose deprecating all the modifications and reverting the ECMAScript engine to its original specification of RegExp, before adding something new on it. But the ECMAScript engine is the default one of regex. Trying to deprecate some features of it seems to increase difficulty in getting the proposal accepted.

Moreover, while `RegExp` supports UTF-16 only, C++'s `regex` needs to support various character sets. It is a difficult question how Unicode property escapes should be processed in legacy (non-Unicode based) character sets.

Thus, this paper does not touch any part of the current ECMAScript engine and leaves it as is for legacy character sets that use `char` and `wchar_t`; and instead propose introducing a new syntax option being compliant to a recent version of ECMAScript keeping the nature of being locale independent, for use with `char8_t`, `char16_t`, or `char32_t` through the template specialization.

III. Why RegExp of ECMAScript?

Because the ECMAScript specification explains the behavior of `RegExp` through defining closures in detail, there seems to be less room for any undefined or undocumented behaviour to appear. It would be an important factor to the standard.

IV. Scope and Impact on the Standard

The new syntax is implemented through the template specialization for `char8_t`, `char16_t`, and `char32_t`. As of C++20, `<regex>` supports only `char` and `wchar_t` and compiling `basic_regex` with the other types leads to compile time error by the reasons explained in P0169. So, the existing implementations would not be affected by this proposal except `regex_iterator`.

basic_regex

When the first template parameter `charT` of the `basic_regex` class is `char8_t`, `char16_t`, or `char32_t`, its constructors and assign functions are required to interpret an input sequence as UTF-8, UTF-16, or UTF-32 respectively. In addition, `syntax_option_types` other than `icase`, `nosubs`, `optimize`, and `multiline` are disabled. An input sequence is always interpreted assuming that **ECMAScript2019** is set.

These three specializations are not required necessarily to be implemented separately. It is expected that typical implementations use an internal iterator class template that translates a sequence of UTF-8, UTF-16, or UTF-32 to a sequence of Unicode code points and construct a finite state machine by parsing that translated sequence in their common base class.

One obstacle to implementing in such a way is the expression `\xHH` where `H` is a hexadecimal digit. In the spec of ECMAScript this expression is defined to represent a code unit. However, appearance of an isolated code unit in a UTF-8 sequence requires special treatment, because unlike in UTF-16 and UTF-32, an isolated code unit in UTF-8 cannot be converted to any code point when its value is in `0x80-0xff` inclusive. (This does not matter in ECMAScript, since it supports UTF-16 only.)

To simplify things, in the proposed **ECMAScript2019** syntax, the expression `\xHH` is changed to represent a code point value. This is the one and only modification to the spec of ECMAScript.

(If the committee does not like the inconsistency with the meaning of C++'s own `\xHH`, the previous paragraph will be changed to "... use of the expression `\xHH` is disabled". RegExp has `\uHHHH` and `\u{H...}` for representing a code point. Removing support for `\xHH` is not likely to cause inconvenience.)

regex_traits

`basic_regex<char8_t>`, `basic_regex<char16_t>`, and `basic_regex<char32_t>` must be locale independent. It means that these specializations have to construct an internal finite state machine only based on the Unicode code point values that are translated from the input sequence and may not use `regex_traits`.

Thus, `regex_traits` does not need to be specialized for `char8_t`, `char16_t`, and `char32_t`.

Particularly, when `basic_regex` is used with `char8_t`, `char16_t`, or `char32_t`, case-folding may not be performed using `regex_traits<charT>::translate_nocase()`, but performed as defined in the ECMAScript specification.

Algorithm functions (regex_match, regex_search, and regex_replace)

As these take an instance of the `basic_regex` class as a parameter, they get `charT` as a template parameter. So, they also can be implemented in a way similar to `basic_regex` using the template specialization.

regex_match and regex_search

The proposed **ECMAScript2019** syntax supports lookbehind assertions, which none of the existing six engines of C++'s regex has support for. When performing a lookbehind assertion, the algorithm function reads the input sequence backwards. This raises a new issue.

When a user wants to find all the portions that some regular expression matches against some input sequence, the user will call `regex_search` against the subrange [the end of the previous matched portion of the entire sequence, the end of the entire sequence) while the previous call succeeds. In this case, the subrange [the beginning of the entire sequence, the end of the previous matched portion of the entire sequence) is also a valid range and it is safe for an algorithm function to read a character in that subrange.

However, there is no way at this time to inform an algorithm function about the limit until where it can read backwards for lookbehind.

For example, when a user calls `regex_search` with the regular expression `/(?<\d{2,})ABC123/` ("ABC123" not preceded by two or more digit characters) against "ABC123ABC1234ABC12345", only the first six characters should be matched, because the second and later "ABC123"s are all preceded by two or more digits. But if there is no way to tell `regex_search` about until where it can look-behind, the second and later call to `regex_search` against [end of previous matched subsequence, end of entire sequence) also return "abc123".

The `match_prev_avail` flag is not suitable for this purpose. It only indicates that the preceding one point is a valid iterator position.

As of C++20, both **regex_match** and **regex_search** take as an input sequence 1) two bidirectional iterators that specify [begin, end), 2) const reference to an instance of **std::basic_string**, or 3) a pointer to a null-terminated string. To fix the problem mentioned above, this paper proposes adding new overload functions that take three bidirectional iterators, which specify [begin, end) and the limit of lookbehind, to both **regex_match** and **regex_search**.

This addition is useful only when an algorithm function is used with an instance of **basic_regex** constructed with **char8_t**, **char16_t**, or **char32_t**. If any variant of algorithm functions that takes three bidirectional iterators is called when its **charT** is **char** or **wchar_t**, the third iterator for specifying the limit of lookbehind is simply ignored.

No specialization is proposed for **regex_replace**. It does not do matching by itself but uses **regex_iterator** that calls **regex_search** internally.

match_results

It is preferable that 1) the new function **group()** and its overload functions be added to **match_results** for access to captured sequences by group name, and 2) the member function **format()** be modified to support the replacement text symbol **\$<GROUPNAME>** that was introduced in ES2018.

However, as **match_results** does not take **charT** as a template parameter, it is not easy to implement something specific to the proposed **ECMAScript2019** syntax option through the template specialization.

Thus, in this proposal, the new member function **gname_to_gnumber()** and its overload functions that convert a group name to the group number assigned with it, are added instead into **basic_regex** specialized for **char8_t**, **char16_t**, and **char32_t**.

regex_iterator

regex_iterator is changed to use one variant of **regex_search** that takes three bidirectional iterators mentioned above, instead of the current one that takes two bidirectional iterators. In this proposal this is the only proposed change that requires modifications to the existing implementations.

Whether this change breaks ABI is likely to depend on the efficiency of the compiler. Because non-specialized **regex_search** that takes three bidirectional iterators only calls a variant that takes two bidirectional iterators, **regex_iterator** ends up calling the variant that it calls at present indirectly when the type of its second template parameter is not **char8_t**, **char16_t**, or **char32_t**. So, it is possible that the output of the compiler is unchanged particularly when optimization is enabled.

This change can be avoided by providing six template specializations for **const char(8|16|32)_t *** and **std::u(8|16|32)string::const_iterator**. But the author of this proposal thinks that **regex_iterator** and what depends on it, namely **regex_token_iterator** and **regex_replace**, are supplements to rather than the core of regex and so in this case the simplest way to implement is preferable. Thus, the "modifying" solution is proposed.

The link to a sample implementation based on the method above is shown in the Appendix section of this document.

V. Technical Specifications

1. <regex>

The following changes are proposed:

■ 30.1 General [re.general]

- 2 The following subclauses describe a basic regular expression class template and its traits that can handle char-like (21.1) template arguments, **two** **five** specializations of this class template that handle sequences of **char** **and**, **wchar_t**, **char8_t**, **char16_t**, and **char32_t**, a class template that holds the result of a regular expression match, a series of algorithms that allow a character sequence to be operated upon by a regular expression, **three specializations of this series that handle sequences of char8_t, char16_t, and char32_t**, and two iterator types for enumerating regular expression matches, as summarized in Table 122.

■ 30.4 Header <regex> synopsis [re.syn]

```
// 30.8, class template basic_regex
template<class charT, class traits = regex_traits<charT>> class
basic_regex;

using regex = basic_regex<char>;
using wregex = basic_regex<wchar_t>;
using u8regex = basic_regex<char8_t>;
using u16regex = basic_regex<char16_t>;
using u32regex = basic_regex<char32_t>;

// 30.9, class template sub_match
template<class BidirectionalIterator>
class sub_match;

using csub_match = sub_match<const char*>;
using wsub_match = sub_match<const wchar_t*>;
using u8csub_match = sub_match<const char8_t*>;
using u16csub_match = sub_match<const char16_t*>;
using u32csub_match = sub_match<const char32_t*>;
using ssub_match = sub_match<string::const_iterator>;
using wssub_match = sub_match<wstring::const_iterator>;
using u8ssub_match = sub_match<u8string::const_iterator>;
```

```
using u16ssub_match = sub_match<u16string::const_iterator>;
using u32ssub_match = sub_match<u32string::const_iterator>;
```

```
// 30.10, class template match_results
```

```
template<class BidirectionalIterator,
         class Allocator = allocator<sub_match<BidirectionalIterator>>>
    class match_results;
```

```
using cmatch = match_results<const char*>;
using wcmatch = match_results<const wchar_t*>;
using u8cmatch = match_results<const u8char_t*>;
using u16cmatch = match_results<const u16char_t*>;
using u32cmatch = match_results<const u32char_t*>;
using smatch = match_results<string::const_iterator>;
using wsmatch = match_results<wstring::const_iterator>;
using u8smatch = match_results<u8string::const_iterator>;
using u16smatch = match_results<u16string::const_iterator>;
using u32smatch = match_results<u32string::const_iterator>;
```

```
// 30.11.2, function template regex_match
```

```
template<class BidirectionalIterator, class Allocator, class charT, class
traits>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                     match_results<BidirectionalIterator, Allocator>& m,
                     const basic_regex<charT, traits>& e,
                     regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class charT, class traits>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                     const basic_regex<charT, traits>& e,
                     regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class charT, class Allocator, class traits>
    bool regex_match(const charT* str, match_results<const charT*,
Allocator>& m,
                     const basic_regex<charT, traits>& e,
                     regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class ST, class SA, class Allocator, class charT, class traits>
    bool regex_match(const basic_string<charT, ST, SA>& s,
                     match_results<typename basic_string<charT, ST,
SA>::const_iterator,
                               Allocator>& m,
                     const basic_regex<charT, traits>& e,
                     regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class ST, class SA, class Allocator, class charT, class traits>
    bool regex_match(const basic_string<charT, ST, SA>&&
```

```

        match_results<typename basic_string<charT, ST,
SA>::const_iterator,
                        Allocator>&,
        const basic_regex<charT, traits>&,
        regex_constants::match_flag_type =
regex_constants::match_default) = delete;
template<class charT, class traits>
    bool regex_match(const charT* str,
        const basic_regex<charT, traits>& e,
        regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class ST, class SA, class charT, class traits>
    bool regex_match(const basic_string<charT, ST, SA>& s,
        const basic_regex<charT, traits>& e,
        regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class Allocator, class charT, class
traits>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
        BidirectionalIterator lookbehindlimit,
        match_results<BidirectionalIterator, Allocator>& m,
        const basic_regex<charT, traits>& e,
        regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class charT, class traits>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
        BidirectionalIterator lookbehindlimit,
        const basic_regex<charT, traits>& e,
        regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class Allocator>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
        BidirectionalIterator lookbehindlimit,
        match_results<BidirectionalIterator, Allocator>& m,
        const basic_regex<char8_t>& e,
        regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class Allocator>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
        match_results<BidirectionalIterator, Allocator>& m,
        const basic_regex<char8_t>& e,
        regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class Allocator>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
        BidirectionalIterator lookbehindlimit,
        match_results<BidirectionalIterator, Allocator>& m,

```



```

        const basic_regex<char16_t>& e,
        regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class Allocator>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
        match_results<BidirectionalIterator, Allocator>& m,
        const basic_regex<char16_t>& e,
        regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class Allocator>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
        BidirectionalIterator lookbehindlimit,
        match_results<BidirectionalIterator, Allocator>& m,
        const basic_regex<char32_t>& e,
        regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class Allocator>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
        match_results<BidirectionalIterator, Allocator>& m,
        const basic_regex<char32_t>& e,
        regex_constants::match_flag_type flags =
regex_constants::match_default);

```

// 30.11.3, function template regex_search

```

template<class BidirectionalIterator, class Allocator, class charT, class
traits>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator
last,
        match_results<BidirectionalIterator, Allocator>& m,
        const basic_regex<charT, traits>& e,
        regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class charT, class traits>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator
last,
        const basic_regex<charT, traits>& e,
        regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class charT, class Allocator, class traits>
    bool regex_search(const charT* str,
        match_results<const charT*, Allocator>& m,
        const basic_regex<charT, traits>& e,
        regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class charT, class traits>
    bool regex_search(const charT* str,
        const basic_regex<charT, traits>& e,

```

```

        regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class ST, class SA, class charT, class traits>
    bool regex_search(const basic_string<charT, ST, SA>& s,
                      const basic_regex<charT, traits>& e,
                      regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class ST, class SA, class Allocator, class charT, class traits>
    bool regex_search(const basic_string<charT, ST, SA>& s,
                      match_results<typename basic_string<charT, ST,
SA>::const_iterator,
                      Allocator>& m,
                      const basic_regex<charT, traits>& e,
                      regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class ST, class SA, class Allocator, class charT, class traits>
    bool regex_search(const basic_string<charT, ST, SA>&&,
                      match_results<typename basic_string<charT, ST,
SA>::const_iterator,
                      Allocator>&,
                      const basic_regex<charT, traits>&,
                      regex_constants::match_flag_type
                        = regex_constants::match_default) = delete;
template<class BidirectionalIterator, class Allocator, class charT, class
traits>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator
last,
                      BidirectionalIterator lookbehindlimit,
                      match_results<BidirectionalIterator, Allocator>& m,
                      const basic_regex<charT, traits>& e,
                      regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class charT, class traits>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator
last,
                      BidirectionalIterator lookbehindlimit,
                      const basic_regex<charT, traits>& e,
                      regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class Allocator>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator
last,
                      BidirectionalIterator lookbehindlimit,
                      match_results<BidirectionalIterator, Allocator>& m,
                      const basic_regex<char8_t>& e,
                      regex_constants::match_flag_type flags =
regex_constants::match_default);

```

```

template<class BidirectionalIterator, class Allocator>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator
last,
                    match_results<BidirectionalIterator, Allocator>& m,
                    const basic_regex<char8_t>& e,
                    regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class Allocator>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator
last,
                    BidirectionalIterator lookbehindlimit,
                    match_results<BidirectionalIterator, Allocator>& m,
                    const basic_regex<char16_t>& e,
                    regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class Allocator>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator
last,
                    match_results<BidirectionalIterator, Allocator>& m,
                    const basic_regex<char16_t>& e,
                    regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class Allocator>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator
last,
                    BidirectionalIterator lookbehindlimit,
                    match_results<BidirectionalIterator, Allocator>& m,
                    const basic_regex<char32_t>& e,
                    regex_constants::match_flag_type flags =
regex_constants::match_default);
template<class BidirectionalIterator, class Allocator>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator
last,
                    match_results<BidirectionalIterator, Allocator>& m,
                    const basic_regex<char32_t>& e,
                    regex_constants::match_flag_type flags =
regex_constants::match_default);

```

// 30.12.1, class template regex_iterator

```

template<class BidirectionalIterator,
        class charT = typename
iterator_traits<BidirectionalIterator>::value_type,
        class traits = regex_traits<charT>>
    class regex_iterator;

using cregex_iterator = regex_iterator<const char*>;
using wcregex_iterator = regex_iterator<const wchar_t*>;
using u8cregex_iterator = regex_iterator<const char8_t*>;

```

```

using u16cregex_iterator = regex_iterator<const char16_t*>;
using u32cregex_iterator = regex_iterator<const char32_t*>;
using sregex_iterator = regex_iterator<string::const_iterator>;
using wsregex_iterator = regex_iterator<wstring::const_iterator>;
using u8sregex_iterator = regex_iterator<u8string::const_iterator>;
using u16sregex_iterator = regex_iterator<u16string::const_iterator>;
using u32sregex_iterator = regex_iterator<u32string::const_iterator>;

// 30.12.2, class template regex_token_iterator
template<class BidirectionalIterator,
         class charT = typename
iterator_traits<BidirectionalIterator>::value_type,
         class traits = regex_traits<charT>>
class regex_token_iterator;

using cregex_token_iterator = regex_token_iterator<const char*>;
using wcregex_token_iterator = regex_token_iterator<const wchar_t*>;
using u8cregex_token_iterator = regex_token_iterator<const char8_t*>;
using u16cregex_token_iterator = regex_token_iterator<const char16_t*>;
using u32cregex_token_iterator = regex_token_iterator<const char32_t*>;
using sregex_token_iterator = regex_token_iterator<string::const_iterator>;
using wsregex_token_iterator =
regex_token_iterator<wstring::const_iterator>;
using u8sregex_token_iterator =
regex_token_iterator<u8string::const_iterator>;
using u16sregex_token_iterator =
regex_token_iterator<u16string::const_iterator>;
using u32sregex_token_iterator =
regex_token_iterator<u32string::const_iterator>;

namespace pmr {
template<class BidirectionalIterator>
using match_results =
std::match_results<BidirectionalIterator,
                  polymorphic_allocator<sub_match<BidirectionalIterator>>>;

using cmatch = match_results<const char*>;
using wcmatch = match_results<const wchar_t*>;
using u8cmatch = match_results<const char8_t*>;
using u16cmatch = match_results<const char16_t*>;
using u32cmatch = match_results<const char32_t*>;
using smatch = match_results<string::const_iterator>;
using wsmatch = match_results<wstring::const_iterator>;
using u8smatch = match_results<u8string::const_iterator>;
using u16smatch = match_results<u16string::const_iterator>;
using u32smatch = match_results<u32string::const_iterator>;
}

```

■ 30.5.1 Bitmask type `syntax_option_type` [re.synopt]

```
namespace std::regex_constants {
    using syntax_option_type = T1;
    inline constexpr syntax_option_type icase = unspecified ;
    inline constexpr syntax_option_type nosubs = unspecified ;
    inline constexpr syntax_option_type optimize = unspecified ;
    inline constexpr syntax_option_type collate = unspecified ;
    inline constexpr syntax_option_type ECMAScript = unspecified ;
    inline constexpr syntax_option_type basic = unspecified ;
    inline constexpr syntax_option_type extended = unspecified ;
    inline constexpr syntax_option_type awk = unspecified ;
    inline constexpr syntax_option_type grep = unspecified ;
    inline constexpr syntax_option_type egrep = unspecified ;
    inline constexpr syntax_option_type multiline = unspecified ;
    inline constexpr syntax_option_type ECMAScript2019 = unspecified ;
    inline constexpr syntax_option_type dotall = unspecified ;
}
```

- 1 The type `syntax_option_type` is an implementation-defined bitmask type (16.4.2.2.4). Setting its elements has the effects listed in Table 124. A valid value of type `syntax_option_type` shall have at most one of the grammar elements `ECMAScript`, `basic`, `extended`, `awk`, `grep`, `egrep`, `ECMAScript2019`, set. If no grammar element is set, the default grammar is `ECMAScript2019` when a value of type `syntax_option_type` is passed to an instance of one of the specializations `basic_regex<char8_t>`, `basic_regex<char16_t>`, and `basic_regex<char32_t>`; otherwise `ECMAScript`.

...

Table 124: `syntax_option_type` effects [tab:re.synopt]

Element	Effect(s) if set
<code>icase</code>	Specifies that matching of regular expressions against a character container sequence shall be performed without regard to case.
<code>nosubs</code>	Specifies that no sub-expressions shall be considered to be marked, so that when a regular expression is matched against a character container sequence, no sub-expression matches shall be stored in the supplied <code>match_results</code> object.
<code>optimize</code>	Specifies that the regular expression engine should pay more attention to the speed with which regular expressions are matched, and less to the speed with which regular expression objects are constructed. Otherwise it has no detectable effect on the program output.

collate	Specifies that character ranges of the form "[a-b]" shall be locale sensitive. This flag has no effect when the ECMAScript2019 engine is selected.
ECMAScript	Specifies that the grammar recognized by the regular expression engine shall be that used by ECMAScript in ECMA-262 third edition, as modified in 30.13. See also: ECMA-262 third edition 15.10 If this flag is passed to an instance of basic_regex<char8_t>, basic_regex<char16_t>, or basic_regex<char32_t>, it shall be interpreted as if no grammar element is set.
basic	Specifies that the grammar recognized by the regular expression engine shall be that used by basic regular expressions in POSIX. See also: POSIX, Base Definitions and Headers, Section 9.3 If this flag is passed to an instance of basic_regex<char8_t>, basic_regex<char16_t>, or basic_regex<char32_t>, it shall be interpreted as if no grammar element is set.
extended	Specifies that the grammar recognized by the regular expression engine shall be that used by extended regular expressions in POSIX. See also: POSIX, Base Definitions and Headers, Section 9.4 If this flag is passed to an instance of basic_regex<char8_t>, basic_regex<char16_t>, or basic_regex<char32_t>, it shall be interpreted as if no grammar element is set.
awk	Specifies that the grammar recognized by the regular expression engine shall be that used by the utility awk in POSIX. If this flag is passed to an instance of basic_regex<char8_t>, basic_regex<char16_t>, or basic_regex<char32_t>, it shall be interpreted as if no grammar element is set.
grep	Specifies that the grammar recognized by the regular expression engine shall be that used by the utility grep in POSIX. If this flag is passed to an instance of basic_regex<char8_t>, basic_regex<char16_t>, or basic_regex<char32_t>, it shall be interpreted as if no grammar element is set.
egrep	Specifies that the grammar recognized by the regular expression engine shall be that used by the utility grep when given the -E option in POSIX. If this flag is passed to an instance of basic_regex<char8_t>, basic_regex<char16_t>, or basic_regex<char32_t>, it shall be interpreted as if no grammar element is set.

multiline	Specifies that <code>^</code> shall match the beginning of a line and <code>\$</code> shall match the end of a line, if the ECMAScript or ECMAScript2019 engine is selected.
ECMAScript2019	Specifies that the grammar recognized by the regular expression engine shall be that used by ECMAScript in ECMA-262 2019 or later, as modified in 30.14. See also: ECMA-262 2019 21.2 If this flag is passed to an instance of basic_regex other than basic_regex<char8_t>, basic_regex<char16_t>, and basic_regex<char32_t>, it shall be interpreted as if no grammar element is set.
dotall	Specifies that <code>.</code> shall match any code point including new-line characters, if the ECMAScript2019 engine is selected.

■ 30.5.2 Bitmask type `match_flag_type` [`re.matchflag`]

Table 125: `regex_constants::match_flag_type` effects when obtaining a match against a character container sequence [`first`, `last`]. [`tab:re.matchflag`]

Element	Effect(s) if set
...	...
format_default	When a regular expression match is to be replaced by a new string, the new string shall be constructed using the rules used by the ECMAScript replace function in ECMA-262 third edition, part 15.5.4.11 <code>String.prototype.replace</code>. In addition, during search and replace operations all non-overlapping occurrences of the regular expression shall be located and replaced, and sections of the input that did not match the expression shall be copied unchanged to the output string.

■ 30.8 Class template `basic_regex` [`re.regex`]

30.8.7 `basic_regex` specializations [`re.regex.special`]

- 1 The header `<regex>` defines three specializations of the class template **basic_regex**: **basic_regex<char8_t>**, **basic_regex<char16_t>**, and **basic_regex<char32_t>**.
- 2 [Note: These specializations are not required necessarily to be implemented separately; typical implementations will use an internal iterator class template that has specializations for **char8_t**,

char16_t, and **char32_t** to translate an input sequence of UTF-8, UTF-16, and UTF-32 respectively to a sequence of Unicode code points, and construct a finite state machine by parsing that translated sequence in a base class shared by these three specializations. —*end note*]

- 3 These specializations shall not use **regex_traits** to construct a internal finite state machine. [*Note*: Particularly, case folding, translating a character prior to comparison without regard to case, shall be performed as defined in ECMA-262 2019 or later, and shall not be performed as defined in **traits::translate_nocase(c)**. —*end note*]

30.8.7.1 class **basic_regex<char8_t>** specializations [re.regex.special.char8_t]

```
namespace std {
    template<>
        class basic_regex<char8_t> {
        public:
            // types
            using value_type = char8_t;
            using traits_type = void;
            using string_type = basic_string<char8_t>;
            using flag_type = regex_constants::syntax_option_type;
            using locale_type = locale;

            // 30.5.1, constants
            static constexpr flag_type  icase = regex_constants::icase;
            static constexpr flag_type  nosubs = regex_constants::nosubs;
            static constexpr flag_type  optimize = regex_constants::optimize;
            static constexpr flag_type  multiline = regex_constants::multiline;
            static constexpr flag_type  ECMAScript2019 =
regex_constants::ECMAScript2019;
            static constexpr flag_type  dotall = regex_constants::dotall;

            // 30.8.7.1.1, construct/copy/destroy
            basic_regex();
            explicit basic_regex(const char8_t* p, flag_type f =
regex_constants::ECMAScript2019);
            basic_regex(const char8_t* p, size_t len, flag_type f =
regex_constants::ECMAScript2019);
            basic_regex(const basic_regex&);
            basic_regex(basic_regex&&) noexcept;
            template<class ST, class SA>
                explicit basic_regex(const basic_string<char8_t, ST, SA>& p,
                                     flag_type f =
regex_constants::ECMAScript2019);
            template<class ForwardIterator>
                basic_regex(ForwardIterator first, ForwardIterator last,
                             flag_type f = regex_constants::ECMAScript2019);
```



```
    basic_regex(initializer_list<char8_t>, flag_type =  
regex_constants::ECMAScript2019);
```

```
    ~basic_regex();
```

```
    basic_regex& operator=(const basic_regex&);  
    basic_regex& operator=(basic_regex&&) noexcept;  
    basic_regex& operator=(const char8_t* ptr);  
    basic_regex& operator=(initializer_list<char8_t> il);  
    template<class ST, class SA>  
        basic_regex& operator=(const basic_string<char8_t, ST, SA>& p);
```

```
    // 30.8.7.1.2, assign
```

```
    basic_regex& assign(const basic_regex& that);  
    basic_regex& assign(basic_regex&& that) noexcept;  
    basic_regex& assign(const char8_t* ptr, flag_type f =  
regex_constants::ECMAScript2019);  
    basic_regex& assign(const char8_t* p, size_t len, flag_type f);  
    template<class string_traits, class A>  
        basic_regex& assign(const basic_string<char8_t, string_traits, A>&  
s,   
                                flag_type f = regex_constants::ECMAScript2019);  
    template<class InputIterator>  
        basic_regex& assign(InputIterator first, InputIterator last,  
                                flag_type f = regex_constants::ECMAScript2019);  
    basic_regex& assign(initializer_list<char8_t>,  
                                flag_type = regex_constants::ECMAScript2019);
```

```
    // 30.8.7.1.3, const operations
```

```
    unsigned mark_count() const;  
    unsigned gname_to_gnumber(const char8_t* p) const;  
    unsigned gname_to_gnumber(const char8_t* p, size_t len) const;  
    template<class string_traits, class A>  
        unsigned gname_to_gnumber(const basic_string<char8_t,  
string_traits, A>& s) const;  
    template<class InputIterator>  
        unsigned gname_to_gnumber(InputIterator first, InputIterator last)  
const;  
    flag_type flags() const;
```

```
    // 30.8.7.1.4, locale
```

```
    locale_type imbue(locale_type loc);  
    locale_type getloc() const;
```

```
    // 30.8.7.1.5, swap
```

```
    void swap(basic_regex&);  
};
```

30.8.7.1.1 Constructors [re.regex.special.char8_t.construct]

basic_regex();

- 1 *Effects:* Constructs an object of class **basic_regex** that does not match any character sequence.

**explicit basic_regex(const char8_t* p, flag_type f =
regex_constants::ECMAScript2019);**

- 2 *Requires:* **p** shall not be a null pointer.
3 *Throws:* **regex_error** if **p** is not a valid regular expression.
4 *Effects:* Constructs an object of class **basic_regex**; the object's internal finite state machine is constructed from the regular expression contained in the array of **char8_t** of length **char_traits<char8_t>::length(p)** whose first element is designated by **p** and whose elements represent a UTF-8 sequence, and interpreted according to the flags **f**.
5 *Ensures:* **flags()** returns **f**. **mark_count()** returns the number of marked sub-expressions within the expression.

**basic_regex(const char8_t* p, size_t len, flag_type f =
regex_constants::ECMAScript2019);**

- 6 *Requires:* **p** shall not be a null pointer.
7 *Throws:* **regex_error** if **p** is not a valid regular expression.
8 *Effects:* Constructs an object of class **basic_regex**; the object's internal finite state machine is constructed from the regular expression contained in the sequence of UTF-8 code units [**p**, **p+len**), and interpreted according the flags specified in **f**.
9 *Ensures:* **flags()** returns **f**. **mark_count()** returns the number of marked sub-expressions within the expression.

basic_regex(const basic_regex& e);

- 10 *Effects:* Constructs an object of class **basic_regex** as a copy of the object **e**.
11 *Ensures:* **flags()** and **mark_count()** return **e.flags()** and **e.mark_count()**, respectively.

basic_regex(basic_regex&& e) noexcept;

- 12 *Effects:* Move constructs an object of class **basic_regex** from **e**.
13 *Ensures:* **flags()** and **mark_count()** return the values that **e.flags()** and **e.mark_count()**, respectively, had before construction. **e** is in a valid state with unspecified value.

**template<class ST, class SA>
explicit basic_regex(const basic_string<char8_t, ST, SA>& s,
flag_type f = regex_constants::ECMAScript2019);**

- 14 *Throws:* **regex_error** if **s** is not a valid regular expression.
- 15 *Effects:* Constructs an object of class **basic_regex**; the object's internal finite state machine is constructed from the regular expression contained in the string **s** whose elements represent a UTF-8 sequence, and interpreted according to the flags specified in **f**.
- 16 *Ensures:* **flags()** returns **f**. **mark_count()** returns the number of marked sub-expressions within the expression.

```
template<class ForwardIterator>
    basic_regex(ForwardIterator first, ForwardIterator last,
                flag_type f = regex_constants::ECMAScript2019);
```

- 17 *Throws:* **regex_error** if the sequence [**first**, **last**) is not a valid regular expression.
- 18 *Effects:* Constructs an object of class **basic_regex**; the object's internal finite state machine is constructed from the regular expression contained in the sequence of UTF-8 code units [**first**, **last**), and interpreted according to the flags specified in **f**.
- 19 *Ensures:* **flags()** returns **f**. **mark_count()** returns the number of marked sub-expressions within the expression.

```
basic_regex(initializer_list<charT> il, flag_type f =
    regex_constants::ECMAScript2019);
```

- 20 *Effects:* Same as **basic_regex(il.begin(), il.end(), f)**.

30.8.7.1.2 Assignment [re.regex.special.char8_t.assign]

```
basic_regex& operator=(const basic_regex& e);
```

- 1 *Effects:* Copies **e** into ***this** and returns ***this**.
- 2 *Ensures:* **flags()** and **mark_count()** return **e.flags()** and **e.mark_count()**, respectively.

```
basic_regex& operator=(basic_regex&& e) noexcept;
```

- 3 *Effects:* Move assigns from **e** into ***this** and returns ***this**.
- 4 *Ensures:* **flags()** and **mark_count()** return the values that **e.flags()** and **e.mark_count()**, respectively, had before assignment. **e** is in a valid state with unspecified value.

```
basic_regex& operator=(const charT* ptr);
```

- 5 *Requires:* **ptr** shall not be a null pointer.
- 6 *Effects:* Returns **assign(ptr)**.

```
basic_regex& operator=(initializer_list<charT> il);
```

- 7 *Effects:* Returns **assign(il.begin(), il.end())**.

```
template<class ST, class SA>
    basic_regex& operator=(const basic_string<charT, ST, SA>& p);
```

8 *Effects:* Returns `assign(p)`.

```
basic_regex& assign(const basic_regex& that);
```

9 *Effects:* Equivalent to: `return *this = that;`

```
basic_regex& assign(basic_regex&& that) noexcept;
```

10 *Effects:* Equivalent to: `return *this = std::move(that);`

```
basic_regex& assign(const charT* ptr, flag_type f =
    regex_constants::ECMAScript2019);
```

11 *Returns:* `assign(string_type(ptr), f)`.

```
basic_regex& assign(const charT* ptr, size_t len, flag_type f =
    regex_constants::ECMAScript2019);
```

12 *Returns:* `assign(string_type(ptr, len), f)`.

```
template<class string_traits, class A>
    basic_regex& assign(const basic_string<charT, string_traits, A>& s,
        flag_type f = regex_constants::ECMAScript2019);
```

13 *Throws:* `regex_error` if `s` is not a valid regular expression.

14 *Returns:* `*this`.

15 *Effects:* Assigns the regular expression contained in the string `s` whose elements represent a UTF-8 sequence, interpreted according the flags specified in `f`. If an exception is thrown, `*this` is unchanged.

16 *Ensures:* If no exception is thrown, `flags()` returns `f` and `mark_count()` returns the number of marked sub-expressions within the expression.

```
template<class InputIterator>
    basic_regex& assign(InputIterator first, InputIterator last,
        flag_type f = regex_constants::ECMAScript2019);
```

17 *Requires:* `InputIterator` shall meet the *Cpp17InputIterator* requirements (23.3.5.2).

18 *Returns:* `assign(string_type(first, last), f)`.

```
basic_regex& assign(initializer_list<charT> il,
    flag_type f = regex_constants::ECMAScript2019);
```

19 *Effects:* Same as `assign(il.begin(), il.end(), f)`.

20 *Returns:* `*this`.

30.8.7.1.3 Constant operations [re.regex.special.char8_t.operations]

```
unsigned mark_count() const;
```

- 1 *Effects:* Returns the number of marked sub-expressions within the regular expression.

```
unsigned gname_to_gnumber(const char8_t* p) const;
```

- 2 *Returns:* `gname_to_gnumber(string_type(p))`.

```
unsigned gname_to_gnumber(const char8_t* p, size_t len) const;
```

- 3 *Returns:* `gname_to_gnumber(string_type(p, len))`.

```
template<class string_traits, class A>  
    unsigned gname_to_gnumber(const basic_string<char8_t, string_traits, A>& s)  
const;
```

- 4 *Throws:* **error_backref** if **s** is an empty string or the marked sub-expression assigned with the group name being identical to the UTF-8 string **s** does not exist within the regular expression.
- 5 *Effects:* Returns the group number of the marked sub-expression assigned with the group name being identical to the UTF-8 string **s**, within the regular expression.

```
template<class InputIterator>  
    unsigned gname_to_gnumber(InputIterator first, InputIterator last) const;
```

- 6 *Requires:* **InputIterator** shall meet the *Cpp17InputIterator* requirements (23.3.5.2).
- 7 *Returns:* `gname_to_gnumber(string_type(first, last))`.

```
flag_type flags() const;
```

- 8 *Effects:* Returns a copy of the regular expression syntax flags that were passed to the object's constructor or to the last call to **assign**.

30.8.7.1.4 Locale [re.regex.special.char8_t.locale]

```
locale_type imbue(locale_type loc);
```

- 1 *Returns:* `locale_type()`.

```
locale_type getloc() const;
```

- 2 *Returns:* `locale_type()`.

30.8.7.1.5 Swap [re.regex.special.char8_t.swap]

```
void swap(basic_regex& e);
```

- 1 *Effects:* Swaps the contents of the two regular expressions.
- 2 *Ensures:* ***this** contains the regular expression that was in **e**, **e** contains the regular expression that was in ***this**.
- 3 *Complexity:* Constant time.

30.8.7.2 class `basic_regex<char16_t>` specializations [re.regex.special.char16_t]

```
namespace std {
    template<>
    class basic_regex<char16_t> {
    public:
        // types
        using value_type = char16_t;
        using traits_type = void;
        using string_type = basic_string<char16_t>;
        using flag_type = regex_constants::syntax_option_type;
        using locale_type = locale;

        // 30.5.1, constants
        static constexpr flag_type icase = regex_constants::icase;
        static constexpr flag_type nosubs = regex_constants::nosubs;
        static constexpr flag_type optimize = regex_constants::optimize;
        static constexpr flag_type multiline = regex_constants::multiline;
        static constexpr flag_type ECMAScript2019 =
regex_constants::ECMAScript2019;
        static constexpr flag_type dotall = regex_constants::dotall;

        // construct/copy/destroy
        basic_regex();
        explicit basic_regex(const char16_t* p, flag_type f =
regex_constants::ECMAScript2019);
        basic_regex(const char16_t* p, size_t len, flag_type f =
regex_constants::ECMAScript2019);
        basic_regex(const basic_regex&);
        basic_regex(basic_regex&&) noexcept;
        template<class ST, class SA>
            explicit basic_regex(const basic_string<char16_t, ST, SA>& p,
                                flag_type f =
regex_constants::ECMAScript2019);
        template<class ForwardIterator>
            basic_regex(ForwardIterator first, ForwardIterator last,
                        flag_type f = regex_constants::ECMAScript2019);
        basic_regex(initializer_list<char16_t>, flag_type =
regex_constants::ECMAScript2019);

        ~basic_regex();
```

```

        basic_regex& operator=(const basic_regex&);
        basic_regex& operator=(basic_regex&&) noexcept;
        basic_regex& operator=(const char16_t* ptr);
        basic_regex& operator=(initializer_list<char16_t> il);
        template<class ST, class SA>
            basic_regex& operator=(const basic_string<char16_t, ST, SA>& p);

    // assign
        basic_regex& assign(const basic_regex& that);
        basic_regex& assign(basic_regex&& that) noexcept;
        basic_regex& assign(const char16_t* ptr, flag_type f =
regex_constants::ECMAScript2019);
        basic_regex& assign(const char16_t* p, size_t len, flag_type f);
        template<class string_traits, class A>
            basic_regex& assign(const basic_string<char16_t, string_traits, A>&
s,
                                flag_type f = regex_constants::ECMAScript2019);
        template<class InputIterator>
            basic_regex& assign(InputIterator first, InputIterator last,
                                flag_type f = regex_constants::ECMAScript2019);
        basic_regex& assign(initializer_list<char16_t>,
                                flag_type = regex_constants::ECMAScript2019);

    // const operations
        unsigned mark_count() const;
        unsigned gname_to_gnumber(const char16_t* p) const;
        unsigned gname_to_gnumber(const char16_t* p, size_t len) const;
        template<class string_traits, class A>
            unsigned gname_to_gnumber(const basic_string<char16_t,
string_traits, A>& s) const;
        template<class InputIterator>
            unsigned gname_to_gnumber(InputIterator first, InputIterator last)
const;
        flag_type flags() const;

    // locale
        locale_type imbue(locale_type loc);
        locale_type getloc() const;

    // swap
        void swap(basic_regex&);
};

```

- 1 Same as the specification of class `basic_regex<char8_t>` specialization, except that the words `char8_t` and UTF-8 that appear in the text are replaced with `char16_t` and UTF-16, respectively.

If saying "Same as the specification of ..." is not appropriate, the previous subclause will be rewritten like [re.regex.special.char8_t].

30.8.7.3 class `basic_regex<char16_t>` specializations [re.regex.special.char32_t]

```
namespace std {
    template<>
        class basic_regex<char32_t> {
        public:
            // types
            using value_type = char32_t;
            using traits_type = void;
            using string_type = basic_string<char32_t>;
            using flag_type = regex_constants::syntax_option_type;
            using locale_type = locale;

            // 30.5.1, constants
            static constexpr flag_type icalse = regex_constants::icalse;
            static constexpr flag_type nosubs = regex_constants::nosubs;
            static constexpr flag_type optimize = regex_constants::optimize;
            static constexpr flag_type multiline = regex_constants::multiline;
            static constexpr flag_type ECMAScript2019 =
regex_constants::ECMAScript2019;
            static constexpr flag_type dotall = regex_constants::dotall;

            // construct/copy/destroy
            basic_regex();
            explicit basic_regex(const char32_t* p, flag_type f =
regex_constants::ECMAScript2019);
            basic_regex(const char32_t* p, size_t len, flag_type f =
regex_constants::ECMAScript2019);
            basic_regex(const basic_regex&);
            basic_regex(basic_regex&&) noexcept;
            template<class ST, class SA>
                explicit basic_regex(const basic_string<char32_t, ST, SA>& p,
                    flag_type f =
regex_constants::ECMAScript2019);
            template<class ForwardIterator>
                basic_regex(ForwardIterator first, ForwardIterator last,
                    flag_type f = regex_constants::ECMAScript2019);
            basic_regex(initializer_list<char32_t>, flag_type =
regex_constants::ECMAScript2019);

            ~basic_regex();

            basic_regex& operator=(const basic_regex&);
            basic_regex& operator=(basic_regex&&) noexcept;
            basic_regex& operator=(const char32_t* ptr);
```



```

        basic_regex& operator=(initializer_list<char32_t> il);
        template<class ST, class SA>
            basic_regex& operator=(const basic_string<char32_t, ST, SA>& p);

    // assign
        basic_regex& assign(const basic_regex& that);
        basic_regex& assign(basic_regex&& that) noexcept;
        basic_regex& assign(const char32_t* ptr, flag_type f =
regex_constants::ECMAScript2019);
        basic_regex& assign(const char32_t* p, size_t len, flag_type f);
        template<class string_traits, class A>
            basic_regex& assign(const basic_string<char32_t, string_traits, A>&
s,
                                flag_type f = regex_constants::ECMAScript2019);
        template<class InputIterator>
            basic_regex& assign(InputIterator first, InputIterator last,
                                flag_type f = regex_constants::ECMAScript2019);
        basic_regex& assign(initializer_list<char32_t>,
                                flag_type = regex_constants::ECMAScript2019);

    // const operations
        unsigned mark_count() const;
        unsigned gname_to_gnumber(const char32_t* p) const;
        unsigned gname_to_gnumber(const char32_t* p, size_t len) const;
        template<class string_traits, class A>
            unsigned gname_to_gnumber(const basic_string<char32_t,
string_traits, A>& s) const;
        template<class InputIterator>
            unsigned gname_to_gnumber(InputIterator first, InputIterator last)
const;
        flag_type flags() const;

    // locale
        locale_type imbue(locale_type loc);
        locale_type getloc() const;

    // swap
        void swap(basic_regex&);
};

```

- 1 Same as the specification of class **basic_regex<char8_t>** specialization, except that the words **char8_t** and UTF-8 that appear in the text are replaced with **char32_t** and UTF-32, respectively.

If saying "Same as the specification of ..." is not appropriate, the previous subclause will be rewritten like [re.regex.special.char8_t].

■ 30.11.2 `regex_match` [re.alg.match]

Addition of variants that take three bidirectional iterators also to non-specialized `regex_match` is for consistency and by analogy to `regex_search`.

```
template<class BidirectionalIterator, class Allocator, class charT, class
traits>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                     BidirectionalIterator lookbehindlimit,
                     match_results<BidirectionalIterator, Allocator>& m,
                     const basic_regex<charT, traits>& e,
                     regex_constants::match_flag_type flags =
regex_constants::match_default);
```

9 *Returns:* `regex_match(first, last, m, e, flags)`.

```
template<class BidirectionalIterator, class Allocator, class charT, class
traits>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                     BidirectionalIterator lookbehindlimit,
                     const basic_regex<charT, traits>& e,
                     regex_constants::match_flag_type flags =
regex_constants::match_default);
```

10 *Returns:* `regex_match(first, last, e, flags)`.

30.11.2.1 `regex_match` specializations [re.alg.match.special]

1 The header `<regex>` defines three specializations of the function template `regex_match` that take as one of parameters an instance of `basic_regex<char8_t>`, `basic_regex<char16_t>`, and `basic_regex<char32_t>`.

2 [Note: These specializations are not required necessarily to be implemented separately; typical implementations will use an internal iterator class template that has specializations for `char8_t`, `char16_t`, and `char32_t` to translate an input sequence of UTF-8, UTF-16, and UTF-32 respectively to a sequence of Unicode code points, and compare that translated sequence with the passed finite state machine in a base function shared by these three specializations. —end note]

```
template<class BidirectionalIterator, class Allocator>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                     BidirectionalIterator lookbehindlimit,
                     match_results<BidirectionalIterator, Allocator>& m,
                     const basic_regex<char8_t>& e,
                     regex_constants::match_flag_type flags =
regex_constants::match_default);
```

- 3 *Requires:* The type **BidirectionalIterator** shall meet the *Cpp17BidirectionalIterator* requirements (23.3.5.5).
- 4 *Effects:* Determines whether there is a match between the regular expression **e**, and all of the UTF-8 sequence [**first**, **last**). The iterator **lookbehindlimit** is used to specify the limit until where reading the UTF-8 sequence backwards can be performed. If **first** **!=** **lookbehindlimit** then **^** shall match **lookbehindlimit** instead of **first**. The parameter **flags** is used to control how the expression is matched against the UTF-8 sequence. When determining if there is a match, only potential matches that match the entire UTF-8 sequence are considered. Returns **true** if such a match exists, **false** otherwise.
- 5 *Ensures:* **m.ready()** **== true** in all cases. If the function returns **false**, then the effect on parameter **m** is unspecified except that **m.size()** returns **0** and **m.empty()** returns **true**. Otherwise the effects on parameter **m** are given in Table 129.

```
template<class BidirectionalIterator, class Allocator>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                    match_results<BidirectionalIterator, Allocator>& m,
                    const basic_regex<char8_t>& e,
                    regex_constants::match_flag_type flags =
regex_constants::match_default);
```

- 6 *Returns:* **regex_match(first, last, first, m, e, flags)**.

```
template<class BidirectionalIterator, class Allocator>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                    BidirectionalIterator lookbehindlimit,
                    match_results<BidirectionalIterator, Allocator>& m,
                    const basic_regex<char16_t>& e,
                    regex_constants::match_flag_type flags =
regex_constants::match_default);
```

- 7 *Requires:* The type **BidirectionalIterator** shall meet the *Cpp17BidirectionalIterator* requirements (23.3.5.5).
- 8 *Effects:* Determines whether there is a match between the regular expression **e**, and all of the UTF-16 sequence [**first**, **last**). The iterator **lookbehindlimit** is used to specify the limit until where reading the UTF-16 sequence backwards can be performed. If **first** **!=** **lookbehindlimit** then **^** shall match **lookbehindlimit** instead of **first**. The parameter **flags** is used to control how the expression is matched against the UTF-16 sequence. When determining if there is a match, only potential matches that match the entire UTF-16 sequence are considered. Returns **true** if such a match exists, **false** otherwise.
- 9 *Ensures:* **m.ready()** **== true** in all cases. If the function returns **false**, then the effect on parameter **m** is unspecified except that **m.size()** returns **0** and **m.empty()** returns **true**. Otherwise the effects on parameter **m** are given in Table 129.

```
template<class BidirectionalIterator, class Allocator>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                     match_results<BidirectionalIterator, Allocator>& m,
                     const basic_regex<char16_t>& e,
                     regex_constants::match_flag_type flags =
regex_constants::match_default);
```

10 *Returns:* `regex_match(first, last, first, m, e, flags)`.

```
template<class BidirectionalIterator, class Allocator>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                     BidirectionalIterator lookbehindlimit,
                     match_results<BidirectionalIterator, Allocator>& m,
                     const basic_regex<char32_t>& e,
                     regex_constants::match_flag_type flags =
regex_constants::match_default);
```

11 *Requires:* The type **BidirectionalIterator** shall meet the *Cpp17BidirectionalIterator* requirements (23.3.5.5).

12 *Effects:* Determines whether there is a match between the regular expression **e**, and all of the UTF-32 sequence **[first, last)**. The iterator **lookbehindlimit** is used to specify the limit until where reading the UTF-32 sequence backwards can be performed. If **first != lookbehindlimit** then $\wedge$ shall match **lookbehindlimit** instead of **first**. The parameter **flags** is used to control how the expression is matched against the UTF-32 sequence. When determining if there is a match, only potential matches that match the entire UTF-32 sequence are considered. Returns **true** if such a match exists, **false** otherwise.

13 *Ensures:* **m.ready() == true** in all cases. If the function returns **false**, then the effect on parameter **m** is unspecified except that **m.size()** returns **0** and **m.empty()** returns **true**. Otherwise the effects on parameter **m** are given in Table 129.

```
template<class BidirectionalIterator, class Allocator>
    bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                     match_results<BidirectionalIterator, Allocator>& m,
                     const basic_regex<char32_t>& e,
                     regex_constants::match_flag_type flags =
regex_constants::match_default);
```

14 *Returns:* `regex_match(first, last, first, m, e, flags)`.

■ 30.11.3 `regex_search` [re.alg.search]

Addition of variants that take three bidirectional iterators also to non-specialized `regex_search` is for `regex_iterator` and consistency.

```
template<class BidirectionalIterator, class Allocator, class charT, class
traits>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
                      BidirectionalIterator lookbehindlimit,
                      match_results<BidirectionalIterator, Allocator>& m,
                      const basic_regex<charT, traits>& e,
                      regex_constants::match_flag_type flags =
regex_constants::match_default);
```

9 *Returns:* `regex_search(first, last, m, e, flags)`.

```
template<class BidirectionalIterator, class Allocator, class charT, class
traits>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
                      BidirectionalIterator lookbehindlimit,
                      const basic_regex<charT, traits>& e,
                      regex_constants::match_flag_type flags =
regex_constants::match_default);
```

10 *Returns:* `regex_search(first, last, e, flags)`.

30.11.3.1 `regex_search` specializations [re.alg.search.special]

1 The header `<regex>` defines three specializations of the function template `regex_search` that take as one of parameters an instance of `basic_regex<char8_t>`, `basic_regex<char16_t>`, and `basic_regex<char32_t>`.

2 [Note: These specializations are not required necessarily to be implemented separately; typical implementations will use an internal iterator class template that has specializations for `char8_t`, `char16_t`, and `char32_t` to translate an input sequence of UTF-8, UTF-16, and UTF-32 respectively to a sequence of Unicode code points, and compare that translated sequence with the passed finite state machine in a base function shared by these three specializations. —end note]

```
template<class BidirectionalIterator, class Allocator>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
                      BidirectionalIterator lookbehindlimit,
                      match_results<BidirectionalIterator, Allocator>& m,
                      const basic_regex<char8_t>& e,
                      regex_constants::match_flag_type flags =
regex_constants::match_default);
```

3 *Requires:* Type `BidirectionalIterator` shall meet the *Cpp17BidirectionalIterator* requirements (23.3.5.5).

4 *Effects:* Determines whether there is some sub-sequence within the UTF-8 sequence `[first, last)` that matches the regular expression `e`. The iterator `lookbehindlimit` is used to specify the limit

until where reading the UTF-8 sequence backwards can be performed. If **first != lookbehindlimit** then ^ shall match **lookbehindlimit** instead of **first**. The parameter **flags** is used to control how the expression is matched against the UTF-8 sequence. Returns **true** if such a sequence exists, **false** otherwise.

- 5 *Ensures:* **m.ready() == true** in all cases. If the function returns **false**, then the effect on parameter **m** is unspecified except that **m.size()** returns **0** and **m.empty()** returns **true**. Otherwise the effects on parameter **m** are given in Table 130.

```
template<class BidirectionalIterator, class Allocator>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
                      match_results<BidirectionalIterator, Allocator>& m,
                      const basic_regex<char8_t>& e,
                      regex_constants::match_flag_type flags =
regex_constants::match_default);
```

- 6 *Returns:* **regex_search(first, last, first, m, e, flags)**.

```
template<class BidirectionalIterator, class Allocator>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
                      BidirectionalIterator lookbehindlimit,
                      match_results<BidirectionalIterator, Allocator>& m,
                      const basic_regex<char16_t>& e,
                      regex_constants::match_flag_type flags =
regex_constants::match_default);
```

- 7 *Requires:* Type **BidirectionalIterator** shall meet the *Cpp17BidirectionalIterator* requirements (23.3.5.5).

- 8 *Effects:* Determines whether there is some sub-sequence within the UTF-16 sequence [**first, last**) that matches the regular expression **e**. The iterator **lookbehindlimit** is used to specify the limit until where reading the UTF-16 sequence backwards can be performed. If **first != lookbehindlimit** then ^ shall match **lookbehindlimit** instead of **first**. The parameter **flags** is used to control how the expression is matched against the UTF-16 sequence. Returns **true** if such a sequence exists, **false** otherwise.

- 9 *Ensures:* **m.ready() == true** in all cases. If the function returns **false**, then the effect on parameter **m** is unspecified except that **m.size()** returns **0** and **m.empty()** returns **true**. Otherwise the effects on parameter **m** are given in Table 130.

```
template<class BidirectionalIterator, class Allocator>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
                      match_results<BidirectionalIterator, Allocator>& m,
                      const basic_regex<char16_t>& e,
                      regex_constants::match_flag_type flags =
regex_constants::match_default);
```

- 10 *Returns:* **regex_search(first, last, first, m, e, flags)**.

```
template<class BidirectionalIterator, class Allocator>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
                      BidirectionalIterator lookbehindlimit,
                      match_results<BidirectionalIterator, Allocator>& m,
                      const basic_regex<char32_t>& e,
                      regex_constants::match_flag_type flags =
regex_constants::match_default);
```

- 11 *Requires:* Type **BidirectionalIterator** shall meet the *Cpp17BidirectionalIterator* requirements (23.3.5.5).
- 12 *Effects:* Determines whether there is some sub-sequence within the UTF-32 sequence [**first**, **last**) that matches the regular expression **e**. The iterator **lookbehindlimit** is used to specify the limit until where reading the UTF-32 sequence backwards can be performed. If **first != lookbehindlimit** then **^** shall match **lookbehindlimit** instead of **first**. The parameter **flags** is used to control how the expression is matched against the UTF-32 sequence. Returns **true** if such a sequence exists, **false** otherwise.
- 13 *Ensures:* **m.ready() == true** in all cases. If the function returns **false**, then the effect on parameter **m** is unspecified except that **m.size()** returns **0** and **m.empty()** returns **true**. Otherwise the effects on parameter **m** are given in Table 130.

```
template<class BidirectionalIterator, class Allocator>
    bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
                      match_results<BidirectionalIterator, Allocator>& m,
                      const basic_regex<char32_t>& e,
                      regex_constants::match_flag_type flags =
regex_constants::match_default);
```

- 14 *Returns:* **regex_search(first, last, first, m, e, flags)**.

■ 30.12 Regular expression iterators [re.iter]

30.12.1.1 Constructors [re.regiter.cnstr]

- 2 *Effects:* Initializes **begin** and **end** to **a** and **b**, respectively, sets **pregex** to **addressof(re)**, sets **flags** to **m**, then calls **regex_search(begin, end, begin, match, *pregex, flags)**. If this call returns **false** the constructor sets ***this** to the end-of-sequence iterator.

30.12.1.4 Increment [re.regiter.incr]

- 3 Otherwise, if the iterator holds a zero-length match, the operator calls:

```
regex_search(start, end, begin, match, *pregex,
             flags | regex_constants::match_not_null |
```

`regex_constants::match_continuous)`

If the call returns `true` the operator returns `*this`. Otherwise the operator increments `start` and continues as if the most recent match was not a zero-length match.

- 4 If the most recent match was not a zero-length match, the operator sets `flags` to `flags | regex_constants::match_prev_avail` and calls `regex_search(start, end, begin, match, *pregex, flags)`. If the call returns `false` the iterator sets `*this` to the end-of-sequence iterator. The iterator then returns `*this`.

■ 30.13 Modified ECMAScript regular expression grammar [re.grammar]

- 1 The regular expression grammar recognized by `basic_regex` objects constructed with the ECMAScript flag is that specified by ECMA-262 [third edition](#), except as specified below.
- 14 The behavior of the internal finite state machine representation when used to match a sequence of characters is as described in ECMA-262 [third edition](#). The behavior is modified according to any `match_flag_type` flags (30.5.2) specified when using the regular expression object in one of the regular expression algorithms (30.11). The behavior is also localized by interaction with the traits class template parameter as follows:

See also: ECMA-262 [third edition](#) 15.10

■ 30.14 Modified ECMAScript2019 regular expression grammar [re.grammar2019]

- 1 The following production within the ECMAScript2019 grammar is modified as follows:

CharacterEscape : HexEscapeSequence

Return the numeric value of the code ~~unit~~[point](#) that is the SV of *HexEscapeSequence*.

Others

The undated version of the ECMAScript Specification is added to references.

■ 2 Normative references [intro.refs]

- (1.1) — Ecma International, ECMAScript Language Specification, Standard Ecma-262.

- (1.42) — Ecma International, ECMAScript Language Specification, Standard Ecma-262, third edition, 1999.
- (1.23) — INTERNET ENGINEERING TASK FORCE (IETF). RFC 6557: Procedures for Maintaining the Time Zone Database [online]. Edited by E. Lear, P. Eggert. February 2012 [viewed 2018-03-26]. Available at <https://www.ietf.org/rfc/rfc6557.txt>
- (1.34) — ISO/IEC 2382 (all parts), Information technology — Vocabulary
- (1.45) — ISO 8601:2004, Data elements and interchange formats — Information interchange — Representation of dates and times
- (1.56) — ISO/IEC 9899:2011, Programming languages — C
- (1.67) — ISO/IEC 9945:2003, Information Technology — Portable Operating System Interface (POSIX)
- (1.78) — ISO/IEC 10646, Information technology — Universal Coded Character Set (UCS)
- (1.89) — ISO/IEC 10646-1:1993, Information technology — Universal Multiple-Octet Coded Character Set (UCS) — Part 1: Architecture and Basic Multilingual Plane
- (1.910) — ISO/IEC/IEEE 60559:2011, Information technology — Microprocessor Systems — Floating-Point arithmetic
- (1.4011) — ISO 80000-2:2009, Quantities and units — Part 2: Mathematical signs and symbols to be used in the natural sciences and technology

VI. Relevant Matter

- The author of this document is aware of P1433 Compile Time Regular Expressions. Even in the case that any proposal based on CTRE becomes part of the C++ standard, it is envisioned that need for <regex> remains for situations where a sequence of regular expressions is settled at runtime.

VII. References

- Ecma International, [ECMA-262, 10th edition, June 2019 ECMAScript® 2019 Language Specification](#)
- The Unicode Consortium, *Unicode Character Database*, [CaseFolding.txt](#)

VIII. Appendix

- [Sample implementation](#)

This example can be used with `char8_t`, `char16_t`, `char32_t` only. `char` and `wchar_t` versions are not implemented. If `basic_regex` or an algorithm function is used with a type other than `char8_t`, `char16_t`, and

`char32_t`, `assert(0)` is called. It demonstrates that adding a new syntax option for `char8_t`, `char16_t`, and `char32_t` through the template specialization is a real option.

All the classes and algorithms are declared in `namespace regex_proposal`, instead of `std`.

This proposal needs a presenter. If you like this proposal and can attend face-to-face committee meetings of C++, please contact me.